

AI Image Detection

AMLS SoSe 2026 Exercise Report

Johannes Hammermann, 470138

July 2026

1 Dataset Exploration and Cleaning

The dataset is first explored for simple clues that could reveal the label without requiring the model to understand image style. Then, the images are cleaned to remove these clues and reduce shortcut risk.

1.1 Dataset

The training set contains 29,688 images: 4,948 real images and 24,740 AI-generated images. This is a 1:5 class imbalance. Each of the six original source classes contains 4,948 images; AI sources 1–5 are merged into label 1.

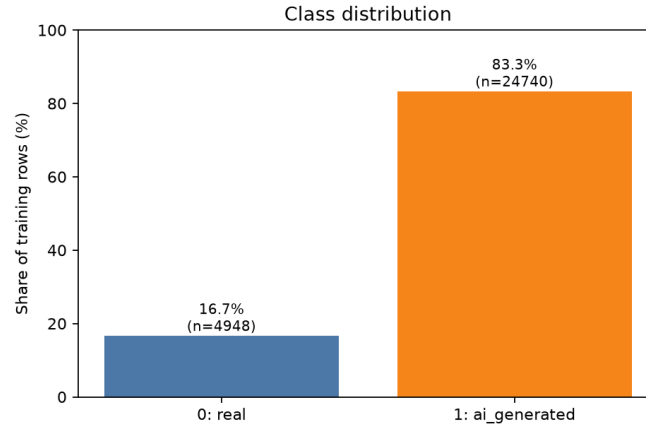


Figure 1: Training class balance.

Image dimensions reveal a clear shortcut risk. The median real image is 640×480 , while the median AI image is 320×320 . Analysis of the distribution of image sizes show that real images are distributed across many common sizes, while AI images are limited to only two different sizes:

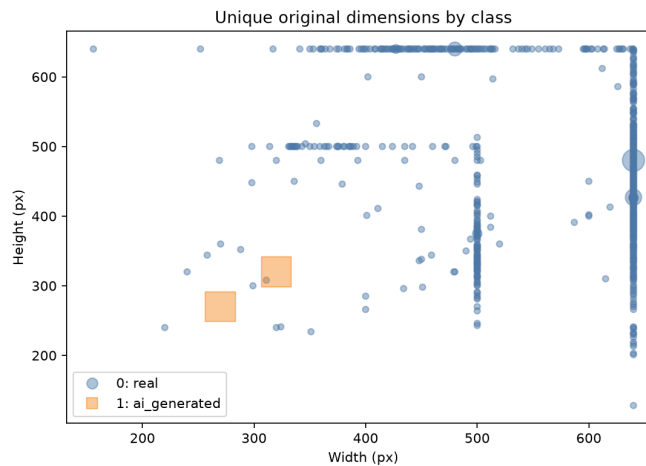


Figure 2: Image width and height by class before cleaning.

AI images are sized either as 320×320 or 270×270 , while the most common real size is 640×480 :

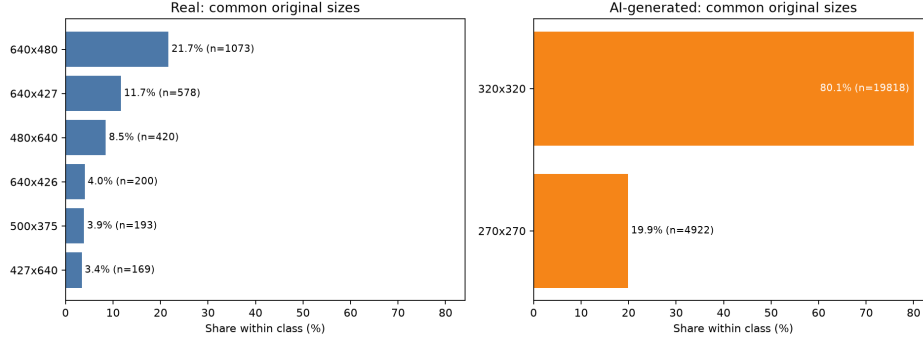


Figure 3: Most common size pairs in the dataset.

To prevent a shortcut during training, the images should be cropped to a common size. To pick a size, I investigate how many images would need to be upsampled, since upscaling would introduce blur, which might also provide a hint in training if the share of blurry images is unevenly distributed across real and AI images.

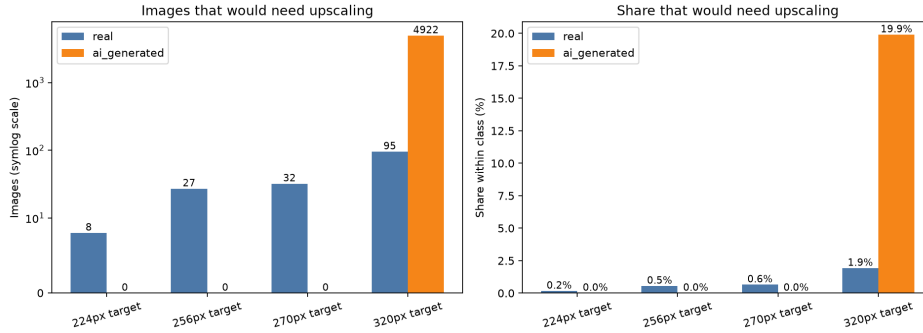


Figure 4: Minimum side thresholds by class.

Since for a 320px target, a much larger share of AI images would need to be upsampled, I choose a 256px target. Compared to 270px, this ensures that all images are downsampled somewhat, which should reduce the risk of downscaling artifacts being a shortcut.

1.2 Deterministic cleaning

Every image is decoded as RGB, cropped to a centred square, and resized to 256×256 using bicubic interpolation. This removes the original dimensions and shape from the model input. The same fixed steps are applied deterministically to every image, with no random augmentation.



Figure 5: Example images after the fixed crop and resize.

2 Task 1.2: Models and Tuning

2.1 Test method and model comparison

The training data is split into multiple sets: the training split is used for model fitting, the calibration split is used to choose the score cut-off, and the validation split is used to measure the final result. All runs use CPU only, fixed random seeds, and at most eight training workers. Two model types are compared against each other:

- Small CNN: three stages learn local visual patterns such as edges and textures, followed by a classification layer.
- Extra Trees: many decision trees vote together; class weights stop the larger AI class from dominating training.

Model	Validation FPR	Validation recall
CNN	0.197	0.603
Extra Trees	0.191	0.826

Table 1: Task 2 model comparison.

The CNN parameters were tuned to improve validation recall from an initial 0.276 (64-pixel, 8-channel, 1-epoch baseline) to the final 0.603 (128-pixel, 32-channel, 5-epoch model). Extra Trees uses a deterministic, class-balanced ensemble of 600 trees on 836 engineered pixel features and shows superior recall and FPR on the same validation set. The Extra Trees model is chosen for Task 1.3.

2.2 Engineered image features

To obtain the engineered image features for the Extra Trees model, each cleaned image is split into an 8×8 grid of 32×32 patches. The 836 features describe local color level and spread, brightness, directional pixel differences, edge residuals, the three lowest bits of each color channel, and JPEG-like block boundaries. The low-bit and maximum-gradient-patch measurements are inspired by LOTA¹. For each patch, the pipeline measures quantities such as RGB mean and variation, luminance, texture strength, and low-bit histograms and entropy. Patch values are then reduced to fixed-length image features using the minimum, maximum, mean, standard deviation, and several percentiles. These summaries describe patterns that occur throughout the image as well as patterns found in only a few patches. In the following chapter, different settings for the numbers of features and other tree parameters are compared.

3 Task 1.3: Robust Training

To measure how well the model performs across different data augmentations independent of calibration and different possible score cut-offs, I use AUC. An AUC of 0.5 is equivalent to random ordering between real and AI images, while 1.0 indicates perfect separation.

3.1 Augmentation and amount of data

To make the model more robust against image changes, I add up to three changed copies of each clean image to the training set. Possible changes are: horizontal flip, 90° rotation, grayscale, contrast, blur, downscale then upscale, pixel noise, or JPEG compression. Each copy receives one operation using seed 42 and can be repeated exactly.

Changed copies	Training rows	Augmented-validation AUC
1	59,376	0.743
2	89,064	0.746
3	118,752	0.748

Table 2: Effect of the number of changed copies on augmented-validation score ordering.

Three changed copies produce the highest AUC in this initial 150-tree comparison, but the difference is small: AUC increases from 0.743 with one copy to 0.748 with three. The final model uses two copies instead. This reduces the training set from 118,752 to 89,064 rows and leaves enough time to increase the ensemble from 150 to 600 trees.

¹Hongsong Wang, Renxi Cheng, Yang Zhang, Chaolei Han, and Jie Gui, “LOTA: Bit-Planes Guided AI-Generated Image Detection,” *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 17246–17255, 2025, open-access paper.

3.2 Tree settings and score cut-off

Features tried per split	Leaf size	Seconds	Clean AUC	Clean recall	Aug. AUC	Aug. recall / FPR
$\sqrt{836} \approx 29$	1	47	0.885	0.756	0.734	0.503 / 0.209
$\sqrt{836} \approx 29$	2	45	0.895	0.792	0.738	0.530 / 0.198
$\sqrt{836} \approx 29$	4	45	0.890	0.791	0.740	0.510 / 0.187
$0.20 \times 836 \approx 167$	2	228	0.899	0.800	0.759	0.553 / 0.214
$0.35 \times 836 \approx 293$	2	387	0.902	0.819	0.757	0.562 / 0.198

Table 3: 150-tree comparison; rows above FPR 0.20 are not valid final choices.

The leaf size is the minimum number of training examples in a final group of a tree. Among the three trials using $\sqrt{836}$ features per split, leaf size 2 gives the best clean AUC (0.895) and the highest augmented recall (0.530), while keeping augmented FPR below 0.20. Leaf size 1 appears to overfit slightly, whereas leaf size 4 smooths the model more strongly and reduces augmented recall.

Extra Trees normally tests a random subset of features at every split so that its trees do not all learn the same rules. The standard choice $\sqrt{836}$ means that about 29 of the 836 features are considered at each split. Increasing this to 20%, or about 167 features, improves augmented score ordering while remaining much cheaper than the 35% setting. This matters for the final 600-tree model.

The final model averages two 300-tree Extra Trees forests with leaf size 2 and 20% of the features tried per split. Both forests use all clean training images and two deterministic augmented copies. The provided augmented calibration images are split into two balanced halves, and each forest is adapted to one half. Each image is scored by the forest that was adapted to the other half, so the model never evaluates an image that it fitted. The predictions of the two forests are then averaged.

Extra Trees model	Clean FPR	Clean recall	Aug. FPR	Aug. recall
Task 1.2	0.191	0.826	0.310	0.572
Task 1.3	0.160	0.823	0.198	0.605

Table 4: Final results on both validation sets.

The Task 1.3 model satisfies the FPR requirement on both validation sets and reaches the target of at least 60% augmented recall. On clean validation, FPR improves from 0.191 to 0.160 while recall changes only slightly from 0.826 to 0.823.

Recall on augmented validation differs by generator: 0.660 for Stable Diffusion 2.1, 0.636 for SDXL, 0.511 for Stable Diffusion 3, 0.695 for DALL-E 3, and 0.524 for Midjourney. Stable Diffusion 3 and Midjourney are therefore the largest remaining weak points. A complete cold Task 1.3 run, starting without Task 1.3 feature or model caches and including augmentation, feature extraction, fitting, calibration, evaluation, and model storage, takes 1,190 seconds on the test machine. This leaves 610 seconds of margin within the 1,800-second limit; exact runtime still depends on hardware. The saved model occupies about 846 MB, and end-to-end prediction for 100 images takes 10.5 seconds on CPU.

4 Task 1.4: Explanations and Errors

4.1 Method

Tree SHAP is used to explain how the Extra Trees models produce their AI scores. It assigns each feature a contribution: a positive contribution raises the AI score, while a negative contribution lowers it. These contributions are added to a reference score. Here, that reference is the expected model output calculated from the numbers of training examples that followed the different tree paths. The reference and all contributions reproduce the predicted score; the largest numerical error observed in this check was below 6×10^{-10} . Tree SHAP is designed to calculate these contributions efficiently and exactly for tree ensembles.²

Since showing all 836 features separately would make the plot unreadable, I combine features that measure the same type of image property into 12 groups. Each heatmap contains the five most confident examples from each outcome: false positive, false negative, true positive, and true negative. The final tile in each block is the mean of its five examples.

²Scott M. Lundberg et al., “From local explanations to global understanding with explainable AI for trees,” *Nature Machine Intelligence*, vol. 2, pp. 56–67, 2020, doi:10.1038/s42256-019-0138-9.

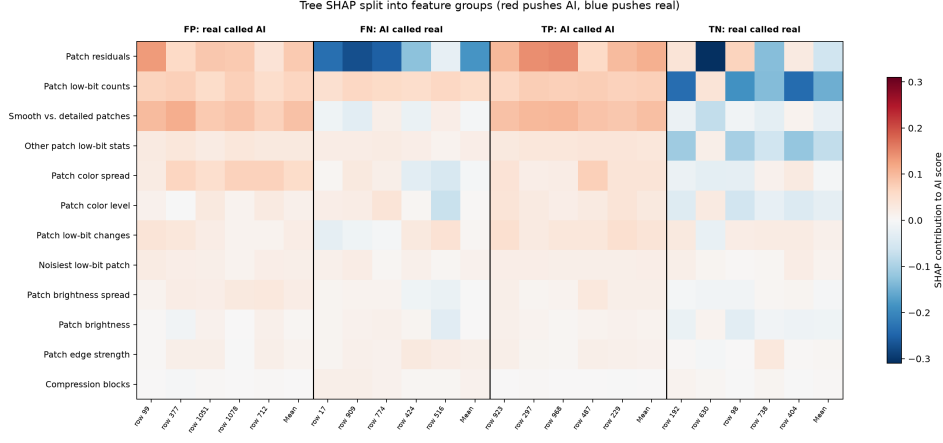


Figure 6: Task 1.2 feature-group contributions for 20 clean-validation examples. Red raises the AI score and blue lowers it; each Mean tile averages the five preceding examples.

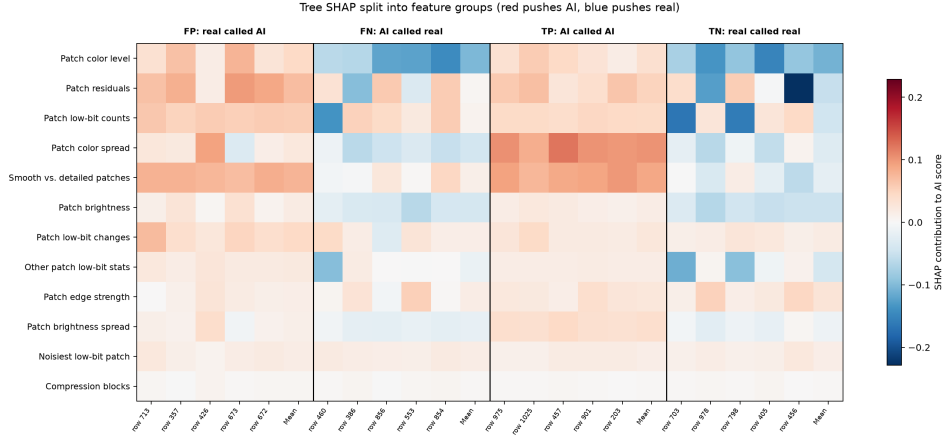


Figure 7: Task 1.3 feature-group contributions for 20 augmented-validation examples, with one mean column per outcome.

4.2 What the explanations show

For Task 1.2, the selected clean true positives are mainly pushed toward AI by residuals, smooth-versus-detailed patch differences, and low-bit counts. The selected true negatives receive strong pushes toward real from low-bit counts, other low-bit statistics, and residuals. The false negatives are dominated by a large negative residual contribution. This shows why the clean model works well overall, but also how an AI image whose residual pattern resembles a real image can be missed.

Task 1.3 has to handle operations that directly change these measurements. Blur, resizing, noise, JPEG compression, grayscale, and contrast adjustment alter residual, low-bit, color, or brightness statistics. In the augmented true positives, color spread and smooth-versus-detailed patch differences give the largest average positive contributions. However, the selected false negatives receive strong negative contributions from color level, followed by color spread, brightness, and low-bit statistics. The augmentations therefore improve the overall score ordering. However, pixel-level cues still change when an image is altered, and this can lead to errors.

Cross-fitted domain adaptation helps the model learn from the augmented domain without testing it on the same examples. Here, each of the two forests is adapted using one half of the augmented calibration set and scores the other half. Their predictions are then averaged.

To analyze why the model missed many altered AI images, it is examined how the model made its decision for the most confident false negative:

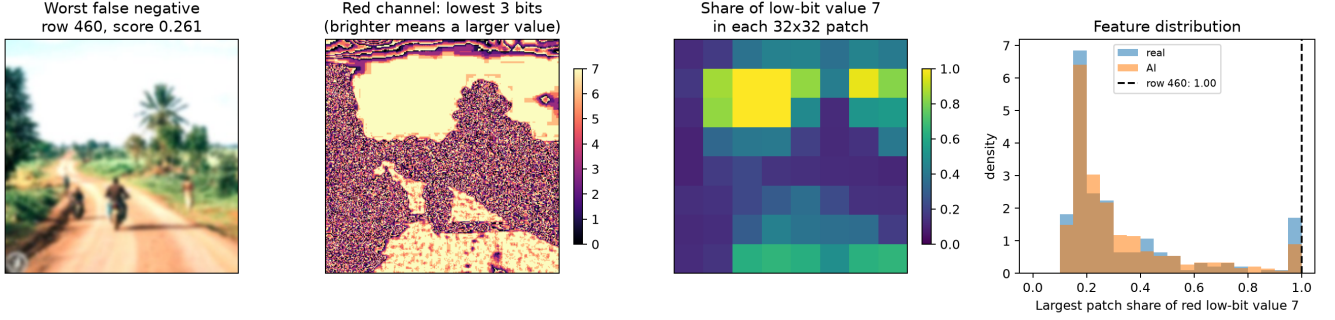


Figure 8: Low-bit diagnostic for the most confident Task 1.3 false negative.

One of the strongest individual contributions for this missed AI image comes from `patch_all_lowbit_red_hist_7_max`. This feature finds the 32×32 patch with the largest share of red pixels whose three lowest bits equal 7. Its value is 1.0 because a nearly white sky patch is entirely at that low-bit value. The model treats this as evidence for a real image. The feature distribution plot, however, shows strong class overlap and an extreme-value spike for both real and AI images, so the feature is not a general AI marker. Low bit planes can contain generation traces, but this case demonstrates how bright, saturated, or compressed regions can confuse such a detector.

4.3 Limits of the explanation

The explanations must be interpreted with care. Similar features can share influence or move credit between one another, so the contribution of a single feature should not be considered in isolation. Combining features into groups also makes the plots easier to read, but two large contributions in opposite directions can cancel each other and become hidden. The selected images are useful case studies rather than a summary of every image in the validation set. The detailed low-bit map is helpful because it connects an abstract feature name to a visible part of the image and reveals a shortcut caused by a bright patch. However, only this one feature is examined visually. It strongly affects the chosen prediction, but the overlapping class distributions show that it is not a reliable general marker of AI-generated images.

5 Summary

This project developed a detector for distinguishing real from AI-generated images. Cropping and resizing every image to 256×256 removed the strong shortcut caused by the different source-image dimensions. Of the two tested approaches, Extra Trees performed better than the small CNN and was therefore used for the final model. The Task 1.3 model achieved a recall of 0.823 at an FPR of 0.160 on clean validation data and a recall of 0.605 at an FPR of 0.198 on augmented validation data, satisfying the required FPR limit in both cases.

The final model combines engineered color, texture, residual, low-bit, and compression-related features with deterministic augmentation and cross-fitted domain adaptation. This improves robustness to altered images while keeping training within the CPU time limit. The explainability investigations show that the detector relies on several complementary pixel-level signals, but also that these signals can change after resizing, compression, or color adjustments. Stable Diffusion 3 and Midjourney remain the most difficult generators, leaving the most room for future improvement.